Intro to Deep Learning

Lecture 03: Classification Losses and Evaluation

Stanislav Don

DS212 - Intro to Deep Learning

Today

By the end of the lecture, you should be able to answer:

1. What is the difference between binary, multiclass, and multilabel classification?
2. What are logits?
3. When do we use sigmoid?
4. When do we use softmax?
5. Why do PyTorch classification losses expect raw logits?
6. How do we inspect model mistakes?

Lecture rhythm: problem type -> output contract -> loss -> metric

Recap From Lecture 02

Training loop:

```
forward -> loss -> backward -> optimizer step
```

Today we focus on the classification loss:

```
model output + target -> classification loss
```

Most classification bugs are caused by mismatched:

```
output shape, target shape, activation, loss
```

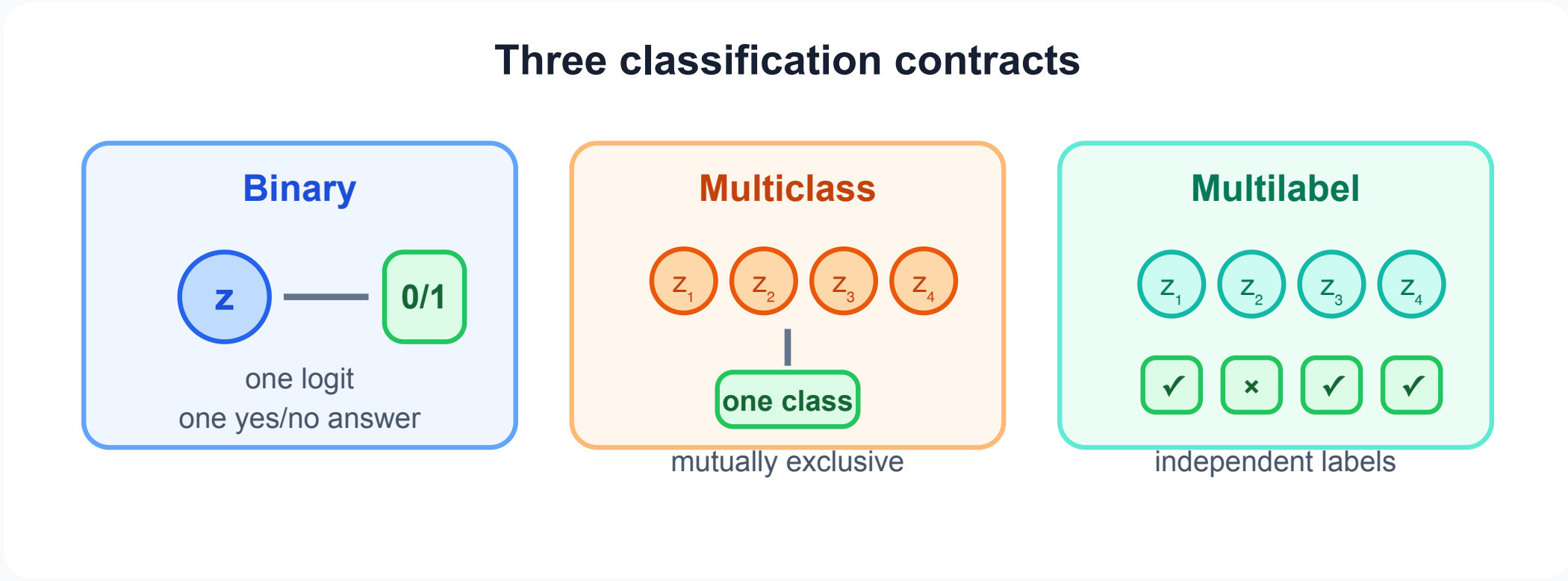
Part 1

Classification Settings

Different questions need different output contracts

Three Classification Settings

Classification is not one problem.



They use different output shapes and losses.

Binary Classification

Question:

```
Does this object belong to class 1?
```

Examples:

- spam / not spam
- fraud / not fraud
- cat / not cat

Typical output:

```
one logit per object
```

Multiclass Classification

Question:

```
Which one class is this object?
```

Examples:

- MNIST digit: 0, 1, ..., 9
- CIFAR-10 class
- news category

Typical output:

```
one logit per class
```

Exactly one class is correct.

Multilabel Classification

Question:

```
Which labels apply to this object?
```

Examples:

- image tags: beach, sunset, people
- movie genres: action, comedy, sci-fi
- medical findings: multiple possible conditions

Typical output:

```
one logit per label
```

Several labels can be correct at once.

Part 2

Logits And Activations

Raw scores become probabilities only when needed

Logits

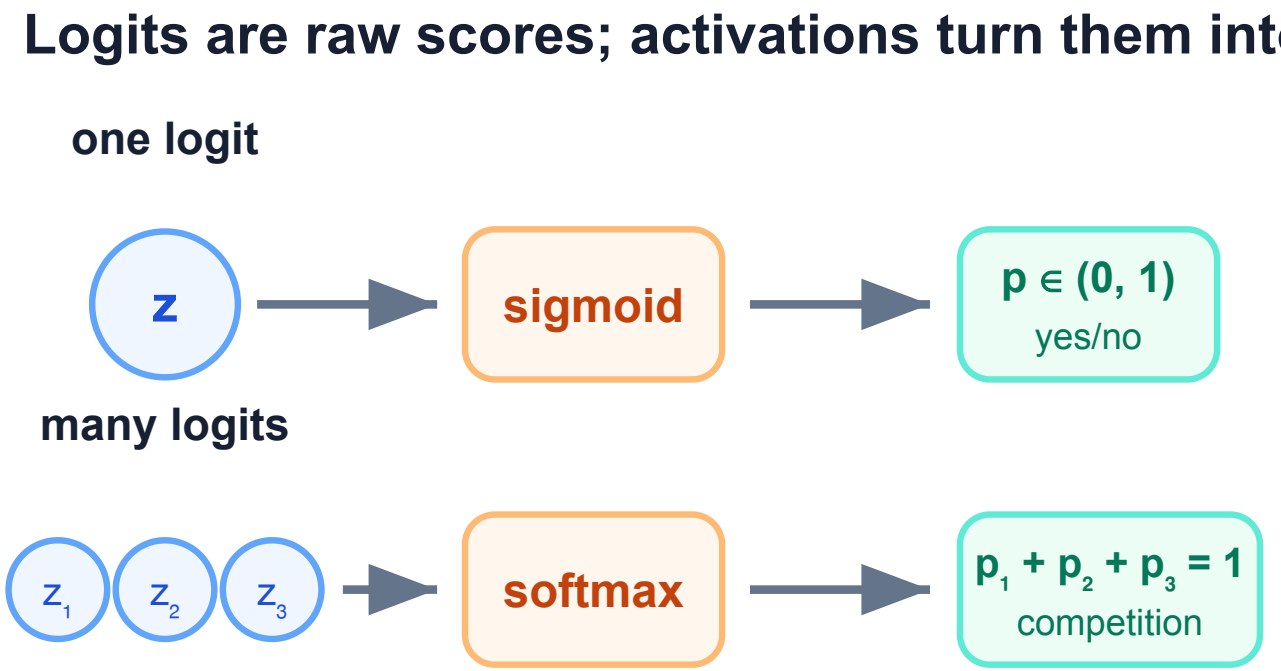
Logits are raw model scores.

Example: `[-2.0, 0.5, 4.1]`

Logits are **not probabilities**:

- they can be negative or larger than 1
- they do not need to sum to 1
- losses often prefer them directly

Sigmoid And Softmax



Sigmoid is for independent yes/no decisions.

Softmax is for mutually exclusive class competition.

Sigmoid

Sigmoid maps one logit to one probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Output:

```
0 < sigmoid(z) < 1
```

Use it when each output is an independent yes/no decision.

Sigmoid For Binary Classification

One logit:

```
z = model(x)
```

Probability of class 1:

```
p = sigmoid(z)
```

Decision rule:

```
predict 1 if p >= 0.5
```

Equivalent:

```
predict 1 if z >= 0
```

Softmax

Softmax converts class logits into probabilities that sum to 1.

$$softmax(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Use it when classes are mutually exclusive.

Example:

```
digit is either 0 or 1 or ... or 9
```

Softmax Intuition

Softmax cares about relative scores.

```
logits = [1, 2, 5]
```

Class 3 receives the highest probability.

If all logits increase by the same constant:

```
[1, 2, 5] -> [101, 102, 105]
```

softmax probabilities stay the same.

Numerically Stable Softmax

Direct exponentials can overflow.

Bad idea:

```
torch.exp(logits)
```

Stable trick:

```
shifted = logits - logits.max(dim=1, keepdim=True).values
probs = torch.exp(shifted) / torch.exp(shifted).sum(dim=1, keepdim=True)
```

This is why we trust library losses.

Why Use Logits In The Loss?

Logits are numerically convenient.

Instead of doing:

```
logits -> probabilities -> log -> loss
```

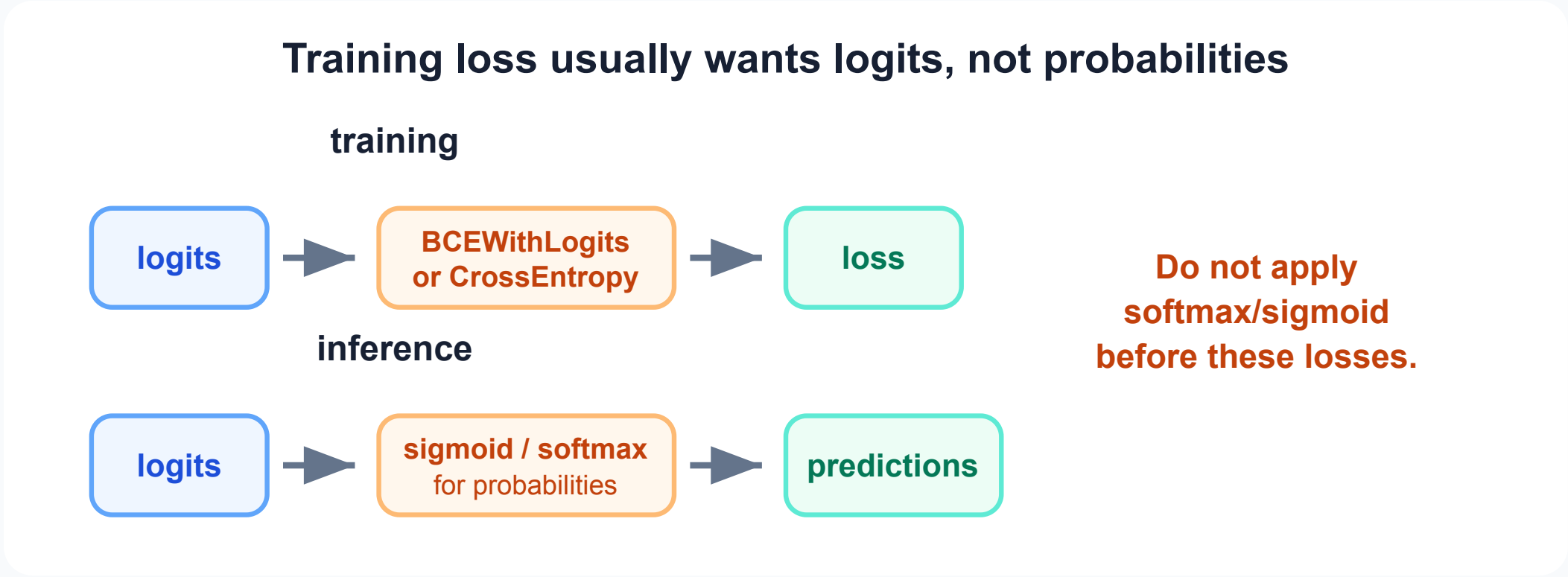
PyTorch losses combine these steps stably.

Examples:

```
nn.BCEWithLogitsLoss()
nn.CrossEntropyLoss()
```

Both expect raw logits.

Training vs Inference



Activation functions are usually for inference or interpretation.

For training, pass logits directly to the matching loss.

Binary Cross-Entropy

For target $y \in \{0, 1\}$ and predicted probability p:

$$BCE(y, p) = -y \log(p) - (1 - y) \log(1 - p)$$

If $y = 1$:

```
loss = -log(p)
```

If $y = 0$:

```
loss = -log(1-p)
```

Binary Loss In PyTorch

Recommended:

```
logits = model(x)
loss_fn = nn.BCEWithLogitsLoss()
loss = loss_fn(logits, targets)
```

Do not put sigmoid inside the model for this loss.

At inference:

```
probs = torch.sigmoid(logits)
preds = (probs >= 0.5)
```

Cross-Entropy

For the correct class y:

$$CE = -\log p_y$$

Meaning:

```
large probability for correct class -> small loss
small probability for correct class -> large loss
```

Cross-entropy punishes confident wrong predictions heavily.

CrossEntropyLoss In PyTorch

For multiclass classification:

```
logits.shape  == (B, C)
targets.shape == (B,)
targets.dtype == torch.long
```

Use:

```
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(logits, targets)
```

Targets are class indices, not one-hot vectors.

What CrossEntropyLoss Does

`nn.CrossEntropyLoss` combines:

```
log_softmax + negative log likelihood
```

Conceptually:

```
log_probs = torch.log_softmax(logits, dim=1)
loss = -log_probs[range(B), targets].mean()
```

So do not apply softmax before `CrossEntropyLoss`.

Common Mistake

Wrong:

```
probs = torch.softmax(logits, dim=1)
loss = nn.CrossEntropyLoss()(probs, targets)
```

Correct:

```
loss = nn.CrossEntropyLoss()(logits, targets)
```

Use softmax only when you need probabilities for interpretation.

Part 4

Output Contracts

Shapes, dtype, loss, and inference must agree

Output And Loss Cheat Sheet

Classification contract table

Problem	Logits	Target	Loss	Inference
Binary	(B) or (B, 1)	float 0/1	BCEWithLogits	sigmoid + threshold
Multiclass	(B, C)	(B) long index	CrossEntropy	argmax / softmax
Multilabel	(B, C)	(B, C) float	BCEWithLogits	sigmoid per label

The shapes tell you what problem you are solving.

Inference Cheat Sheet

Binary:

```
prob = torch.sigmoid(logit)
pred = prob >= 0.5
```

Multiclass:

```
prob = torch.softmax(logits, dim=1)
pred = logits.argmax(dim=1)
```

Multilabel:

```
probs = torch.sigmoid(logits)
preds = probs >= threshold
```

Why Not Softmax For Multilabel?

Softmax forces probabilities to compete:

```
probabilities sum to 1
```

But multilabel classes are not mutually exclusive.

An image can be:

```
beach + sunset + people
```

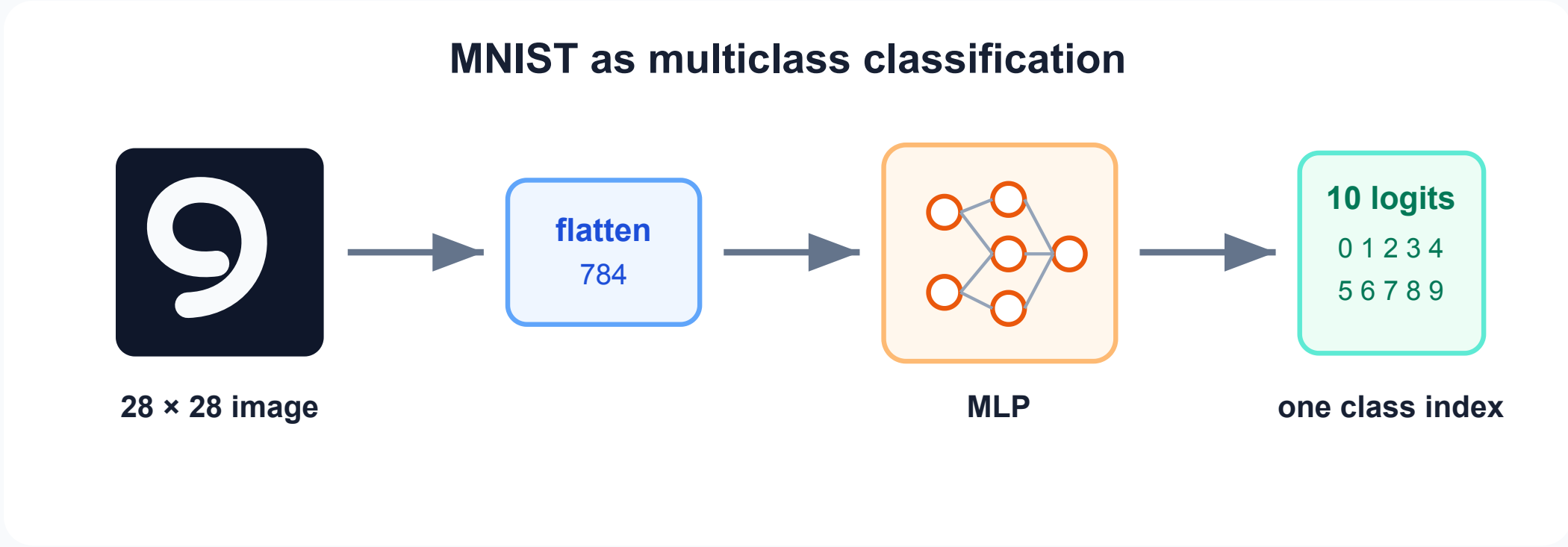
So each label needs its own sigmoid yes/no decision.

Part 5

MNIST And Evaluation

Metrics are summaries; mistakes are evidence

MNIST As Multiclass Classification



For training: `logits.shape == (B, 10)` and `loss_fn = nn.CrossEntropyLoss()`.

MNIST MLP

Simple baseline:

```
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, hidden),
    nn.ReLU(),
    nn.Linear(hidden, 10),
)
```

Output:

```
logits.shape == (B, 10)
```

Accuracy

For multiclass classification:

```
preds = logits.argmax(dim=1)
accuracy = (preds == targets).float().mean()
```

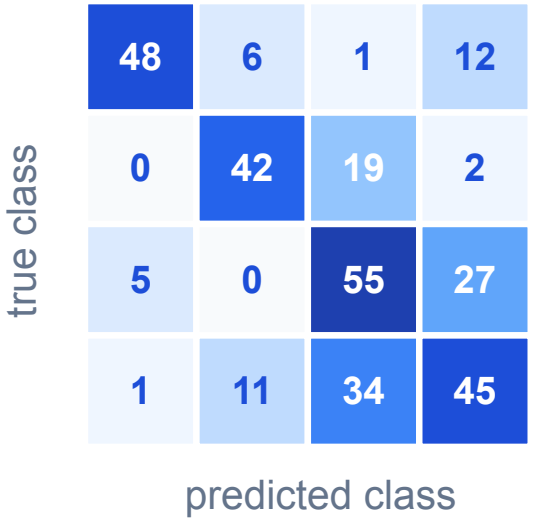
Accuracy is intuitive.

But it can hide important details:

- which classes are confused?
- are mistakes systematic?
- is the dataset imbalanced?

Confusion Matrix

A confusion matrix turns accuracy into questions



Look beyond the diagonal

Which classes are mixed up?

Are errors symmetric?

Which examples should we inspect?

These mistakes are more informative than a single accuracy number.

Error Analysis

After training, inspect wrong predictions.

Ask:

1. Is the label ambiguous?
2. Is the image low quality?
3. Are mistakes concentrated in certain classes?
4. Does the model fail on a specific style?
5. Would more data or a better architecture help?

Looking at mistakes is part of modeling.

Part 6

Metric Warnings

The metric should match the real question

Metric Warning: Imbalanced Data

Accuracy can be misleading.

Example: if 95% of examples are negative, a model that always predicts “negative” gets 95% accuracy.

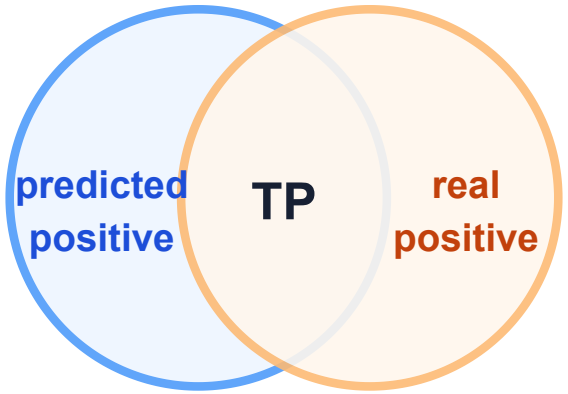
But it learned nothing useful about the positive class.

For imbalanced problems, consider:

`precision`, `recall`, `F1`, `ROC-AUC`, `PR-AUC`

Precision And Recall

Precision and recall answer different questions



Precision

Of predicted positives, how many were correct?

TP / (TP + FP)

Recall

Of real positives, how many did we find?

TP / (TP + FN)

More aggressive positive predictions usually increase recall and decrease precision.

We will revisit this in detection and retrieval.

Practical Checklist

Before training a classifier, decide:

1. Is it binary, multiclass, or multilabel?
2. What shape should logits have?
3. What shape and dtype should targets have?
4. Which loss expects those tensors?
5. Which activation is only for inference?
6. Which metric answers the real question?

This checklist prevents many silent bugs.

Part 7

Seminar Bridge

The contract should become automatic

Mini Check

You have 32 images and 10 mutually exclusive classes.

The model returns:

```
logits.shape == (32, 10)
```

What should the target look like for `CrossEntropyLoss`?

Answer

Targets should be class indices:

```
targets.shape == (32,)
targets.dtype == torch.long
```

Example:

```
targets = tensor([3, 0, 9, 1, ...])
```

Not one-hot vectors.

Mini Check

You have image tags:

```
beach, sunset, people, car, dog
```

Each image can have several tags.

Should you use softmax or sigmoid?

Answer

Use sigmoid.

Each label is an independent yes/no decision.

Output:

```
one logit per label
```

Loss:

```
nn.BCEWithLogitsLoss()
```

What Will Happen In Seminar?

You will:

1. map classification settings to correct losses
2. implement stable softmax
3. verify cross-entropy equals log-softmax + NLL
4. train an MNIST MLP baseline
5. compute multiclass accuracy
6. inspect misclassified digits

The goal is not just high accuracy.

The goal is knowing exactly what the model outputs and what the loss expects.

Key Takeaways

1. Logits are raw model scores.
2. Use `BCEWithLogitsLoss` for binary and multilabel classification.
3. Use `CrossEntropyLoss` for multiclass classification.
4. Do not apply sigmoid/softmax before these losses.
5. Target shape and dtype matter.
6. Error analysis is part of evaluation.

Next lecture:

```
generalization, overfitting, regularization, and learning-rate schedules
```